Function Fitting

Contents

2.1 Ingredients for function fitting **2**
2.1.1 Cost functions 3
2.1.2 From KL-divergence to the cross-entropy cost in numbers 9

2.2 Gradient-based learning **11**
2.2.1 Finding the minimum of the training error analytically 11
2.2.2 Gradient descent 12
2.2.3 Gradient calculation: Connectionist neuron 13

2.1 Ingredients for function fitting

Fitting a model to a desired function $y(\underline{\mathbf{x}})$ requires the following:

1. A cost function with the objective to optimize it, often a minimization problem:

$$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \tag{2.1.1}$$

2. A performance measure, a criterion for *model selection*. Specifically, the generalization **error** E^G which is defined as:

$$E^G_{[\underline{\mathbf{w}}]} \;:=\; \langle\, e \,\rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} \;=\; \iint d\underline{\mathbf{x}}\, dy_T\; P(y_T, \underline{\mathbf{x}})\, e(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)) \tag{2.1.2}$$

Because $P(y_T, \underline{\mathbf{x}})$ is not known, we turn to the principle of empirical risk minimization (ERM). According to ERM we can approximate $E^G_{[\underline{\mathbf{w}}]}$ by computing the

empirical average $E^T_{[\underline{\mathbf{w}}]}$ using the available training data:

$$\left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \alpha = 1, \ldots, p$$

. The training error $E^T_{[\underline{\mathbf{w}}]}$ becomes:

$$\text{batch training error:} \quad E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}_{e^{(\alpha)}} \tag{2.1.3}$$

where $e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)$ (or $e^{(\alpha)}$ for brevity) is the cost computed from the prediction for a specific observation $y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})$ and its corresponding label $y_T^{(\alpha)}$. The superscript $^{(\alpha)}$ is used to index a specific point (sample) in the dataset.

3. A model with tunable parameters $\underline{\mathbf{w}}$: connectionist neuron, MLP,...
4. A learning algorithm for finding the set of parameters in our model that will minimize the cost function. This can be done analytically (depending on some conditions) or through an iterative learning algorithm (e.g. gradient-based learning)

2.1.1 Cost functions

A cost function $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$, quantifies the discrepancy between the model's prediction $y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$ and the label y_T which $\underline{\mathbf{x}}$ is assigned to.

Choosing a cost function accounts for

1. the type of problem (i.e. regression vs. classification),
2. how the model is penalized for different types of mistakes it can make such as: small errors are tolerable, large errors are penalized less, etc.

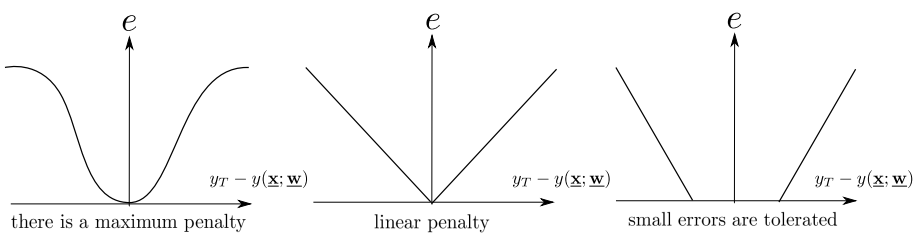
The choice of cost function has a direct effect on how the model learns. For example, we will later see in gradient-based learning how the error function can modulate how fast or slow a model learns.

2.1.1.1 Cost functions for Regression

$$\underbrace{\underline{\mathbf{x}} \in \mathbb{R}^N}_{\text{feature vector}} \longrightarrow \underbrace{y \in \mathbb{R}}_{\substack{\text{scalar}\\ \text{attribute}}}$$

$y_T(\underline{\mathbf{x}})$: true value of attribute
$y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$: predicted value of attribute (e.g. by MLP)

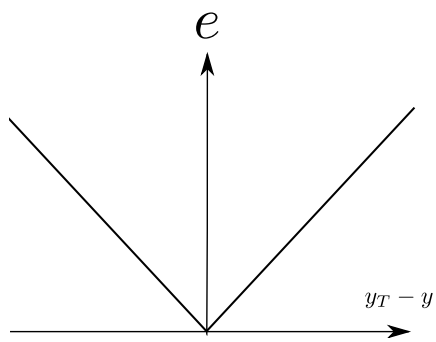
individual cost $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$



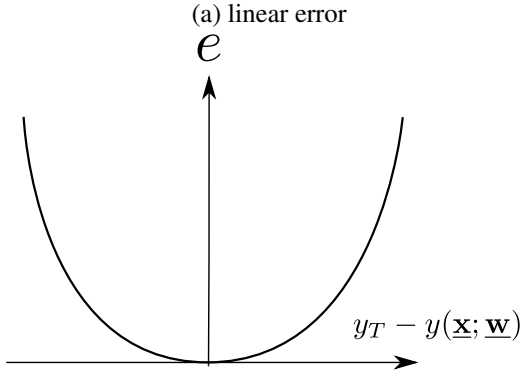
$$e_{\text{quadratic}}\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\Big(y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\Big)^2 \tag{2.1.4}$$

$$e_{\text{linear}}\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \Big|y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\Big| \tag{2.1.5}$$

The purpose of having $\frac{1}{2}$ in Eq. (2.1.4) is for convenience for when we calculate the derivative later. Both the quadratic and linear cost functions are symmetric. They therefore yield the same error regardless of the sign. However, the differences between them are:



(a) linear error



(b) qudratic error

Figure 2.1.1: quadratic vs. linear error

Table 1: Differences between quadratic and linear cost

quadratic	linear
larger error $\rightsquigarrow$ larger penalty (sensitive to outliers)	constant increase in error (more stable, robust to outliers)
converges faster but slow convergence for small errors	constant convergence rate
differentiable everywhere	not differentiable at zero (not a huge problem)
max. likelihood of Gaussian variable	

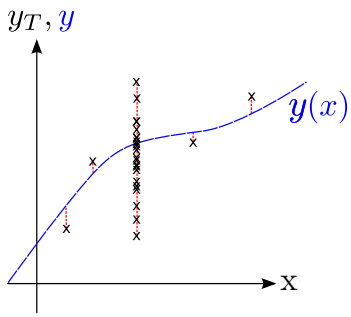
Relation between quadratic error and Gaussian labels

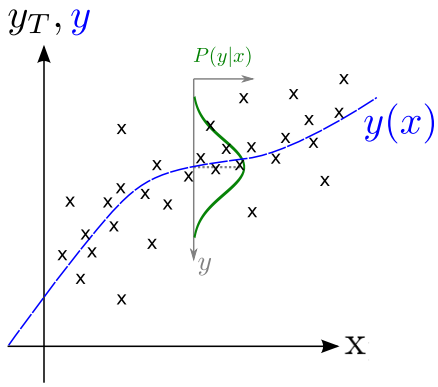
Assume labels y_T are conditionally Gaussian:

$$P(y_T|\underline{\mathbf{x}}) = \mathcal{N}(y_T|\underbrace{y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}_{=\mu}, \sigma^2) \tag{2.1.6}$$

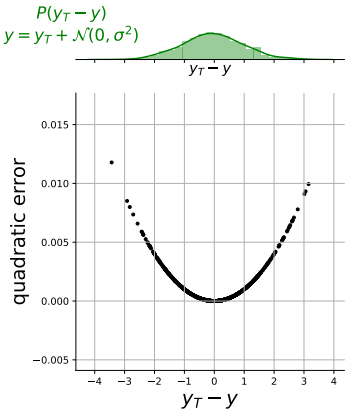
Using the quadratic cost leads to a solution that corresponds to the same solution found from maximizing the (log-)likelihood[1] of the Gaussian labels:

[1] often, the log-likelihood is maximized for computational efficiency as the log replaces the product with a sum.





(a) guassian labels



(b) aggregate density

Figure 2.1.2: quadratic error and guassian labels

$$\underline{\mathbf{w}}^* \in \underset{\underline{\mathbf{w}}}{\operatorname{argmin}}\, E^T_{[\underline{\mathbf{w}}]} \iff \underset{\underline{\mathbf{w}}}{\operatorname{argmax}} \underbrace{\prod_{\alpha=1}^{p} \mathcal{N}(y_T | y(\underline{\mathbf{x}}; \underline{\mathbf{w}}), \sigma^2)}_{\text{likelihood function } \mathcal{L}(\underline{\mathbf{w}})} \tag{2.1.7}$$

$$= \underset{\underline{\mathbf{w}}}{\operatorname{argmin}} \underbrace{[-\ln \mathcal{L}(\underline{\mathbf{w}})]}_{\text{neg. log likelihood}} \tag{2.1.8}$$

This property makes the quadratic cost function the standard choice for regression.

2.1.1.2 Cost function for binary classification

Deriving the cross entropy cost function for binary classification

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p} \tag{2.1.9}$$

with

$$\underline{\mathbf{x}}^{(\alpha)} \in \mathbb{R}^N \quad , \quad y_T^{(\alpha)} \in \{0, 1\}.$$

generated by

$$\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \overset{i.i.d.}{\sim} P_{\text{data}}(\underline{\mathbf{x}}, y) \tag{2.1.10}$$

Model:

MLP, scalar output interpreted as probability that $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = 1$. In other words the probability of $\underline{\mathbf{x}}$ belonging to class '1' (the positive class):

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) =: P_{\text{model}}(y = 1|\underline{\mathbf{x}}; \underline{\mathbf{w}}) \quad \text{and} \quad P_{\text{model}}(y = 0|\underline{\mathbf{x}}; \underline{\mathbf{w}}) = 1 - y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) \tag{2.1.11}$$

for the "positive" and "negative"/"other" class respectively.

The Kullback-Leibler divergence

$\mathrm{D_{KL}}(P||Q)$ is a measure of how much a probability distribution P differs from another distribution Q.

$$\mathrm{D_{KL}}\left(P(z)|| \, Q(z) \right) := \int dz \, P(z) \ln \left(\frac{P(z)}{Q(z)} \right) \tag{2.1.12}$$

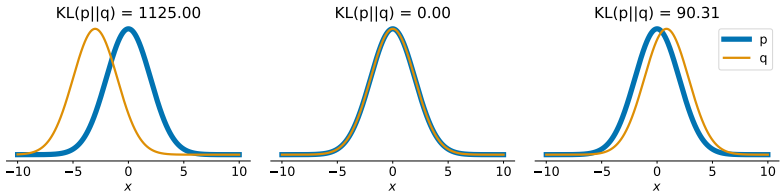


Figure 2.1.3: The KL divergence for comparing two distributions.

Properties of the KL-divergence:

- $\mathrm{D_{KL}} = 0$ iff. both distribution are *identical*,
- otherwise, $\mathrm{D_{KL}} > 0$.

! Does not qualify as a metric because it is not symmetric. For different distributions P and Q:

$$\mathrm{D_{KL}}(P||Q) \neq \mathrm{D_{KL}}(Q||P) \tag{2.1.13}$$

From KL-divergence to binary cross entropy loss:

$$\mathrm{D_{KL}}\left(P_{\text{data}}(\underline{\mathbf{x}}, y) \, || \, P_{\text{model}}(\underline{\mathbf{x}}, y) \right) = \int\int d\underline{\mathbf{x}} \, dy P_{\text{data}}(\underline{\mathbf{x}}, y) \ln \left(\frac{P_{\text{data}}(\underline{\mathbf{x}}, y)}{P_{\text{model}}(\underline{\mathbf{x}}, y)} \right) \tag{2.1.14}$$

Integrating w.r.t. $y \mathrel{\widehat{=}}$ sum over the set of all possible values for y, which is $\{0, 1\}$ in this case:

$$\mathrm{D_{KL}}\left(P_{\text{data}}(\underline{\mathbf{x}}, y) \,||\, P_{\text{model}}(\underline{\mathbf{x}}, y) \right) \tag{2.1.15}$$

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} P_{\text{data}}(\underline{\mathbf{x}}, y) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}, y)}{P_{\text{model}}(\underline{\mathbf{x}}, y)} \right) \tag{2.1.16}$$

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}_{\substack{=P_{\text{data}}(\underline{\mathbf{x}},y) \\ \text{(Chain rule)}}} \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}{\underbrace{P_{\cancel{\text{model}}}(\underline{\mathbf{x}})}_{\substack{\text{discriminative model} \\ \text{not a generative model} \\ \text{replace with } P_{\text{data}}(\underline{\mathbf{x}})}} P_{\text{model}}(y|\underline{\mathbf{x}})} \right) \tag{2.1.17}$$

$$= \int_{\mathbb{R}^N} d\underline{\mathbf{x}} \sum_{y\in\{0,1\}} \underbrace{P_{\text{data}}(\underline{\mathbf{x}})}_{\text{indep. of } y} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln\left(\frac{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y|\underline{\mathbf{x}})}{P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{model}}(y|\underline{\mathbf{x}})} \right) \tag{2.1.18}$$

$$= \underbrace{\int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{data}}(y|\underline{\mathbf{x}})]}_{\text{indep. of model parameters } \underline{\mathbf{w}}} \tag{2.1.19}$$

$$- \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \underbrace{\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}})]}_{\substack{\textbf{cross entropy} \text{ between} \\ \text{data \& model distributions given } \underline{\mathbf{x}}}} \tag{2.1.20}$$

ERM for computing cross entropy loss using the training data:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} -\left(\sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}}^{(\alpha)})] \right) \tag{2.1.21}$$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} -\Big(P_{\text{data}}(y=1|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=1|\underline{\mathbf{x}}^{(\alpha)})] \tag{2.1.22}$$

$$+ P_{\text{data}}(y=0|\underline{\mathbf{x}}^{(\alpha)}) \ln[P_{\text{model}}(y=0|\underline{\mathbf{x}}^{(\alpha)})] \Big) \tag{2.1.23}$$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} -\left(y_T^{(\alpha)} \cdot \ln[y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})] + (1 - y_T^{(\alpha)}) \cdot \ln[1 - y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})] \right) \tag{2.1.24}$$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} \left(- y_T^{(\alpha)} \cdot \ln[y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})] - (1 - y_T^{(\alpha)}) \cdot \ln[1 - y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})] \right) \tag{2.1.25}$$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right) \tag{2.1.26}$$

Extendable to multi-class cross entropy alongside *softmax* normalization.

Table 2: Cost functions & output layers

task	data $\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\}_{\alpha=1}^{p}$	output layer	cost function $e := e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$
scalar regression	$y_T \in \mathbb{R}$	linear $y = \underline{\mathbf{w}}^{LL-1\top}\underline{\mathbf{s}}^{L-1}$	quadratic error $e = \frac{1}{2}\left(y_T - y\right)^2$
multivariate linear regression	$\underline{\mathbf{y}}_T \in \mathbb{R}^M$	linear $y_k = \underline{\mathbf{w}}_k^{LL-1\top}\underline{\mathbf{s}}^{L-1}$	squared Euclidean distance $e = \frac{1}{2}\|\underline{\mathbf{y}}_T - \underline{\mathbf{y}}\|_2^2$
binary classification	$y_T \in \{0,1\}, \{-1,1\}$	logistic sigmoid $y = \frac{1}{1+exp(-h_1^L)}$ or $y = \tanh(h_1^L)$	cross entropy $e = -y_T \ln[y]$ $-(1-y_T)\ln[1-y]$
multi-class classification with M classes	$\underline{\mathbf{y}}_T \in \{0,1\}^M$ 1-out-of-M encoding/ 1-hot-encoding e.g. $\underline{\mathbf{y}}_T = (0,0,1,0)^\top$ $\widehat{=}$ class 3 out of 4	softmax $y_k = \frac{exp(h_k^L)}{\sum_{k=1}^M exp(h_k^L)}$ $\Rightarrow \sum_{k=1}^M y_k = 1$	cross entropy (multi-class case) $e = -\sum_{k=1}^M y_{Tk}$ $\cdot \ln[y_k\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)]$

2.1.2 From KL-divergence to the cross-entropy cost in numbers

2.1.2.1 First: The KL-divergence in numbers

$$\mathrm{D_{KL}}\left(P_{\text{data}}(\underline{\mathbf{x}}, y) \,||\, P_{\text{model}}(\underline{\mathbf{x}}, y) \right) \tag{2.1.27}$$

$$= \underbrace{\int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{data}}(y|\underline{\mathbf{x}})]}_{\text{indep. of } \underline{\mathbf{w}} \Rightarrow \text{ treat as const. } = c} \tag{2.1.28}$$

$$- \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}})] \tag{2.1.29}$$

$$= c - \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \sum_{y\in\{0,1\}} P_{\text{data}}(y|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y|\underline{\mathbf{x}})] \tag{2.1.30}$$

$$= c - \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) \Big[P_{\text{data}}(y=0|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y=0|\underline{\mathbf{x}})] \tag{2.1.31}$$

$$+ P_{\text{data}}(y=1|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y=1|\underline{\mathbf{x}})] \Big] \tag{2.1.32}$$

$$= c - \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y=0|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y=0|\underline{\mathbf{x}})] \tag{2.1.33}$$

$$- \int_{\mathbb{R}^N} d\underline{\mathbf{x}}\, P_{\text{data}}(\underline{\mathbf{x}}) P_{\text{data}}(y=1|\underline{\mathbf{x}}) \ln[P_{\text{model}}(y=1|\underline{\mathbf{x}})] \tag{2.1.34}$$

1. **Correctly** classifiying a single positive sample:
   A positive sample implies:
   - $P_{\text{data}}(y=0|\underline{\mathbf{x}}) = 0.01$
   - $P_{\text{data}}(y=1|\underline{\mathbf{x}}) = 0.99$

   A **correct** classification implies:
   - $P_{\text{model}}(y=0|\underline{\mathbf{x}}) = 0.2$
   - $P_{\text{model}}(y=1|\underline{\mathbf{x}}) = 0.8$

   Plugging it into the equation above. Only a single sample, so no integration:

$$\mathrm{D_{KL}} = c - P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.01 \cdot \ln(0.2) - P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.99 \cdot \ln(0.8) \tag{2.1.35}$$

$$= c + P_{\text{data}}(\underline{\mathbf{x}}) \cdot [-0.01 \cdot \ln(0.2) - 0.99 \cdot \ln(0.8)] \tag{2.1.36}$$

$$= c + P_{\text{data}}(\underline{\mathbf{x}}) \cdot [0.016 + 0.221] \tag{2.1.37}$$

$$= c + P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.237 \tag{2.1.38}$$

2. **Misclassifiying** a single positive sample:
   A positive sample implies:
   - $P_{\text{data}}(y=0|\underline{\mathbf{x}}) = 0.01$
   - $P_{\text{data}}(y=1|\underline{\mathbf{x}}) = 0.99$

   A **misclassification** implies:
   - $P_{\text{model}}(y=0|\underline{\mathbf{x}}) = 0.7$
   - $P_{\text{model}}(y=1|\underline{\mathbf{x}}) = 0.3$

Plugging it into the equation above. Only a single sample, so no integration:

$$\begin{aligned}
\mathrm{D_{KL}} &= c \; - P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.01 \cdot \ln(0.7) - P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.99 \cdot \ln(0.3) && (2.1.39)\\
&= c \; + P_{\text{data}}(\underline{\mathbf{x}}) \cdot [-0.01 \cdot \ln(0.7) - 0.99 \cdot \ln(0.3)] && (2.1.40)\\
&= c \; + P_{\text{data}}(\underline{\mathbf{x}}) \cdot [0.004 + 1.19] && (2.1.41)\\
&= c \; + P_{\text{data}}(\underline{\mathbf{x}}) \cdot 1.194 && (2.1.42)
\end{aligned}$$

$$\begin{aligned}
\mathrm{D_{KL}}(\text{correct classification}) &< \mathrm{D_{KL}}(\text{misclassification}) && (2.1.43)\\
c \; + P_{\text{data}}(\underline{\mathbf{x}}) \cdot 0.237 &< c \; + P_{\text{data}}(\underline{\mathbf{x}}) \cdot 1.194 && (2.1.44)
\end{aligned}$$

2.1.2.2 Second: binary cross-entropy in numbers

binary cross-entropy for an indivudal sample:

$$e(y_T, y(\underline{\mathbf{x}}; \underline{\mathbf{w}})) = -y_T \cdot \ln[y(\underline{\mathbf{x}}; \underline{\mathbf{w}})] - (1 - y_T) \cdot \ln[1 - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})] \quad (2.1.45)$$

1. **Correctly** classifiying a single positive sample:
   A positive sample implies:
   - $y_T = 1$

   A **correct** classification implies:
   - $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = 0.8$

   Plugging it into Eq.(2.1.45) above:

$$\begin{aligned}
e(1, 0.8) &= -\,1 \cdot \ln[0.8] - (1 - 1) \cdot \ln[1 - 0.8] && (2.1.46)\\
&= -\ln[0.8] - (0) \cdot \ln[0.2] && (2.1.47)\\
&= 0.223 && (2.1.48)
\end{aligned}$$

2. **Misclassifying** a single positive sample:
   A positive sample implies:
   - $y_T = 1$

   A **wrong** classification implies:
   - $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = 0.3$

   Plugging it into Eq.(2.1.45) above:

$$\begin{aligned}
e(1, 0.3) &= -\,1 \cdot \ln[0.3] - (1 - 1) \cdot \ln[1 - 0.3] && (2.1.49)\\
&= -\ln[0.3] - (0) \cdot \ln[0.7] && (2.1.50)\\
&= 1.204 && (2.1.51)
\end{aligned}$$

$$\begin{aligned}
e(\text{correct}) &< e(\text{wrong}) && (2.1.52)\\
0.223 &< 1.204 && (2.1.53)
\end{aligned}$$

2.2 Gradient-based learning

Training error E^T for the training set $\left\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\right\}$:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p}\sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right) \tag{2.2.1}$$

The objective is to minimize the training error w.r.t. the model parameters $\underline{\mathbf{w}}$. That is

$$E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \quad \Rightarrow \quad \underline{\mathbf{w}}^* = \operatorname*{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} \tag{2.2.2}$$

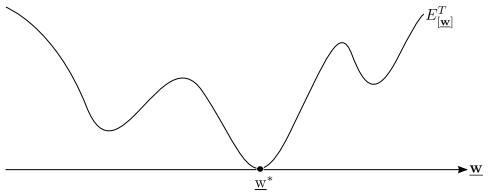


Figure 2.2.1: Training error with minimum at $\underline{\mathbf{w}}^*$

Q 2.2.1: *What is the strategy for finding $\underline{\mathbf{w}}^*$ analytically?*

2.2.1 Finding the minimum of the training error analytically

Example with a simple connectionist neuron model with parameters

$$\underline{\mathbf{w}} = (w_0, w_1, \ldots, w_N)^\top$$

The strategy for finding the minimum of a function $E^T_{[\underline{\mathbf{w}}]}$ analytically is as follows:

1. Compute the gradient w.r.t. $\underline{\mathbf{w}}$ by taking the first partial derivatives w.r.t to each component in the vector $\underline{\mathbf{w}}$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} = \left(\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_0}, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_1}, \ldots, \frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_N}\right)^\top \tag{2.2.3}$$

   The gradient $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}}$ has the same dimensionality as $\underline{\mathbf{w}}$.

2. Set the gradient to zero: $\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \underline{\mathbf{w}}} \stackrel{!}{=} \underline{\mathbf{0}}$
3. Solve for $\underline{\mathbf{w}}$ to find extrema.
4. Select solution corresponding to global minimum.

Caveat: Closed-form solution infeasible for complex models such as MLPs.

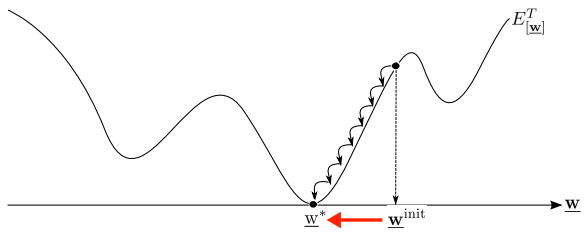


Figure 2.2.2: Minimizing the training error iteratively via gradient descent

2.2.2 Gradient descent

Learning from gradient descent is an alternate approach for finding the minimum of a function when the closed-form solution is not available.

For our connectionist neuron example learning by gradient descent would update each weight iteratively using:

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{gradient vector}}} \tag{2.2.4}$$

with $j = 0, \ldots, N$ $\underline{\mathbf{w}}^{\text{init}}$ from Fig. 2.2.2 is a random guess of where the solution might be and going against the gradient will potentially move us closer to the minimum.

The learning step (learning rate) η modulates the magnitude of our update. η can be treated as a constant but we will also see how the value of η can change over time, i.e. $\eta(t)$.

Q 2.2.2: *Why do we <u>subtract</u> the gradient from $w_j(t)$?* The gradient de-

scribes the slope. Adding it will move us upwards and potentially maximize our function. Gradient-based learning with the intention of maximizing some function is referred to as hill climbing or gradient *ascent*.

Q 2.2.3: *So what is the downside of using gradient descent?*

- The solution we find heavily depends on the initial position we started from. Indeed, gradient descent will reduce our cost each step but it does not guarantee that it will find the global minimum and will also stop at a local minimum. The slope in both cases is equal to zero.

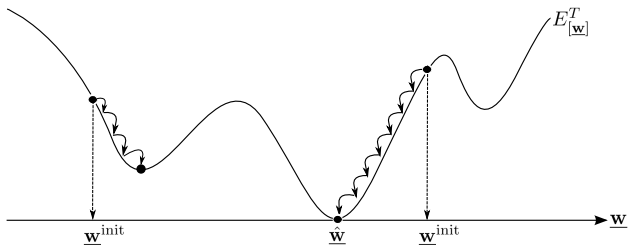


Figure 2.2.3: Gradient descent finds local minima. We therefore denote the solution with $\hat{\underline{\mathbf{w}}}$

2.2.3 Gradient calculation: Connectionist neuron

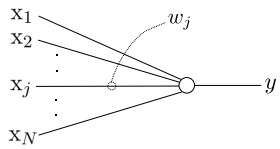


Figure 2.2.4: A connctionist neuron

Knowing that

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e^{(\alpha)}}{\partial w_j} \tag{2.2.5}$$

with $j = 0, \ldots, N$.

The individual cost $e^{(\alpha)}$ is a function of terms that are functions of other terms themselves. Therefore, $\frac{\partial e^{(\alpha)}}{\partial w_j}$ is computed by applying the *chain rule*:

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}} \tag{2.2.6}$$

The first factor represents the first link from applying the chain rule. One recognizes that it only depends on the choice of the cost function. This *error term* is completely independent of the type of model we choose.

Therefore, if the objective were to minimize quadratic error:

$$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2 \tag{2.2.7}$$

it follows:

$$\frac{\partial\, e_{\text{quadr.}}\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})} = -y_T^{(\alpha)} + y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) \tag{2.2.8}$$

The model-specific contribution to the error appears in the second factor.

Continuing with our connectionist neuron from Fig. 2.2.4:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) := f\Big(\sum_{j=0}^{N} w_j x_j\Big) = f\left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}}\right) = f\left(h(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right) \tag{2.2.9}$$

It follows:

$$\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j} = \frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}{\partial h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})} \cdot \frac{\partial h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j} \tag{2.2.10}$$

$$= \underbrace{f'(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}))}_{\substack{\text{depends on}\\ \text{transfer function}}} \cdot \underbrace{\frac{\partial\, \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)}}{\partial w_j}}_{=x_j} \tag{2.2.11}$$